

**Московский государственный технический
университет им. Н. Э. Баумана**

Курс «Технологии машинного обучения»

Отчёт по рубежному контролю №2

«Методы построения моделей машинного обучения.»

Вариант № 17

Выполнил:
Хабленко И. Д.
группа ИУ5-63Б

Проверил:
Гапанюк Ю.Е.

Дата: 09.06.25

Дата:

Подпись:

Подпись:

Москва, 2025 г.

Задание:

Номер варианта: 17

Метод 1: Дерево решений.

Метод 2: Случайный лес.

Номер набора данных, указанного в задаче: 17

(<https://www.kaggle.com/datasets/mathan/fifa-2018-match-statistics/data>)

Задача.

Для заданного набора данных (по Вашему варианту) постройте модели классификации или регрессии (в зависимости от конкретной задачи, рассматриваемой в наборе данных). Для построения моделей используйте методы 1 и 2 (по варианту для Вашей группы). Оцените качество моделей на основе подходящих метрик качества (не менее двух метрик). Какие метрики качества Вы использовали и почему? Какие выводы Вы можете сделать о качестве построенных моделей? Для построения моделей необходимо выполнить требуемую предобработку данных: заполнение пропусков, кодирование категориальных признаков, и т.д.

Ход выполнения:

РК2. Вариант 17.

Задание. Для заданного набора данных (по Вашему варианту) постройте модели классификации или регрессии (в зависимости от конкретной задачи, рассматриваемой в наборе данных). Для построения моделей используйте методы 1 и 2 (по варианту для Вашей группы). Оцените качество моделей на основе подходящих метрик качества (не менее двух метрик). Какие метрики качества Вы использовали и почему? Какие выводы Вы можете сделать о качестве построенных моделей? Для построения моделей необходимо выполнить требуемую предобработку данных: заполнение пропусков, кодирование категориальных признаков, и т.д.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, f1_score, roc_auc_score, classification_report, confusion_matrix
from sklearn.preprocessing import LabelEncoder, StandardScaler
```

✓ 0.0s

Python

```
# Загрузка данных
data = pd.read_csv("FIFA 2018 Statistics.csv")

# Предварительная обработка данных
def preprocess_data(df):
    # Заполнение пропусков
    df.fillna(0, inplace=True)

    # Удаление нерелевантных столбцов
    df = df.drop(["Date", "Own goal Time"], axis=1)

    # Создание новых признаков
    df['Total_Shots'] = df['On-Target'] + df['Off-Target']
    df['Shot_Accuracy'] = np.where(df['Total_Shots'] > 0,
                                   df['On-Target'] / df['Total_Shots'], 0)
    df['Goal_Efficiency'] = np.where(df['On-Target'] > 0,
                                     df['Goal_Scored'] / df['On-Target'], 0)

    return df

data = preprocess_data(data)
```

Python

```

# Кодирование категориальных признаков
label_encoders = {}
categorical_cols = ["Team", "Opponent", "Round", "PSO"]
for col in categorical_cols:
    le = LabelEncoder()
    data[col] = le.fit_transform(data[col])
    label_encoders[col] = le

# Кодирование целевой переменной
le_target = LabelEncoder()
data["Man of the Match"] = le_target.fit_transform(data["Man of the Match"])

# Разделение данных
X = data.drop("Man of the Match", axis=1)
y = data["Man of the Match"]

# Проверка дисбаланса классов
print("Class distribution:\n", y.value_counts(normalize=True))

# Разделение на train/test с стратификацией
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# Масштабирование числовых признаков
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

```

✓ 0.0s

Python

```

Class distribution:
Man of the Match
1    0.5
0    0.5
Name: proportion, dtype: float64

```

```

# Функция для оценки модели
def evaluate_model(model, X_test, y_test, model_name):
    y_pred = model.predict(X_test)
    y_proba = model.predict_proba(X_test)[:, 1] if hasattr(model, "predict_proba") else [0]*len(y_test)

    print(f"\n{model_name} Performance:")
    print("Accuracy:", accuracy_score(y_test, y_pred))
    print("F1-score:", f1_score(y_test, y_pred))
    if hasattr(model, "predict_proba"):
        print("ROC-AUC:", roc_auc_score(y_test, y_proba))
    print("\nClassification Report:")
    print(classification_report(y_test, y_pred))
    print("\nConfusion Matrix:")
    print(confusion_matrix(y_test, y_pred))

```

✓ 0.0s

Python

Дерево решений

Модель 1: Дерево решений

```
dt_params = {
    'max_depth': [3, 5, 7, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'criterion': ['gini', 'entropy']
}

dt_grid = GridSearchCV(DecisionTreeClassifier(random_state=42),
                       dt_params, cv=5, scoring='f1', n_jobs=-1)
dt_grid.fit(X_train_scaled, y_train)
best_dt = dt_grid.best_estimator_

print("\nЛучшие гиперпараметры Дерево решений:", dt_grid.best_params_)
evaluate_model(best_dt, X_test_scaled, y_test, "Decision Tree")
```

✓ 0.5s

Python

Лучшие гиперпараметры Дерево решений: {'criterion': 'entropy', 'max_depth': 3, 'min_samples_leaf': 1, 'min_samples_split': 2}

Decision Tree Performance:

Accuracy: 0.6923076923076923

F1-score: 0.75

ROC-AUC: 0.668639053254438

Classification Report:

	precision	recall	f1-score	support
0	0.86	0.46	0.60	13
1	0.63	0.92	0.75	13
accuracy			0.69	26
macro avg	0.74	0.69	0.68	26
weighted avg	0.74	0.69	0.68	26

Confusion Matrix:

```
[[ 6  7]
 [ 1 12]]
```

Случайный лес

Модель 2: Случайный лес

```
rf_params = {
    'n_estimators': [100, 200, 500],
    'max_depth': [10, 20, 50],
    'min_samples_split': [2, 5],
    'min_samples_leaf': [1, 2],
    'class_weight': ['balanced']
}

rf_grid = GridSearchCV(RandomForestClassifier(random_state=42),
                       rf_params, cv=5, scoring='roc_auc', n_jobs=-1)
rf_grid.fit(X_train_scaled, y_train)
best_rf = rf_grid.best_estimator_

print("\nЛучшие гиперпараметры Случайный лес:", rf_grid.best_params_)
evaluate_model(best_rf, X_test_scaled, y_test, "Random Forest")

# Важность признаков для Random Forest
import matplotlib.pyplot as plt

feature_importance = best_rf.feature_importances_
sorted_idx = np.argsort(feature_importance)[-15:]
pos = np.arange(len(sorted_idx)) + 0.5

plt.figure(figsize=(10, 6))
plt.barh(pos, feature_importance[sorted_idx], align='center')
plt.yticks(pos, X.columns[sorted_idx])
plt.xlabel('Feature Importance')
plt.title('Важность признаков для Random Forest')
plt.show()
```

✓ 8.2s

Python

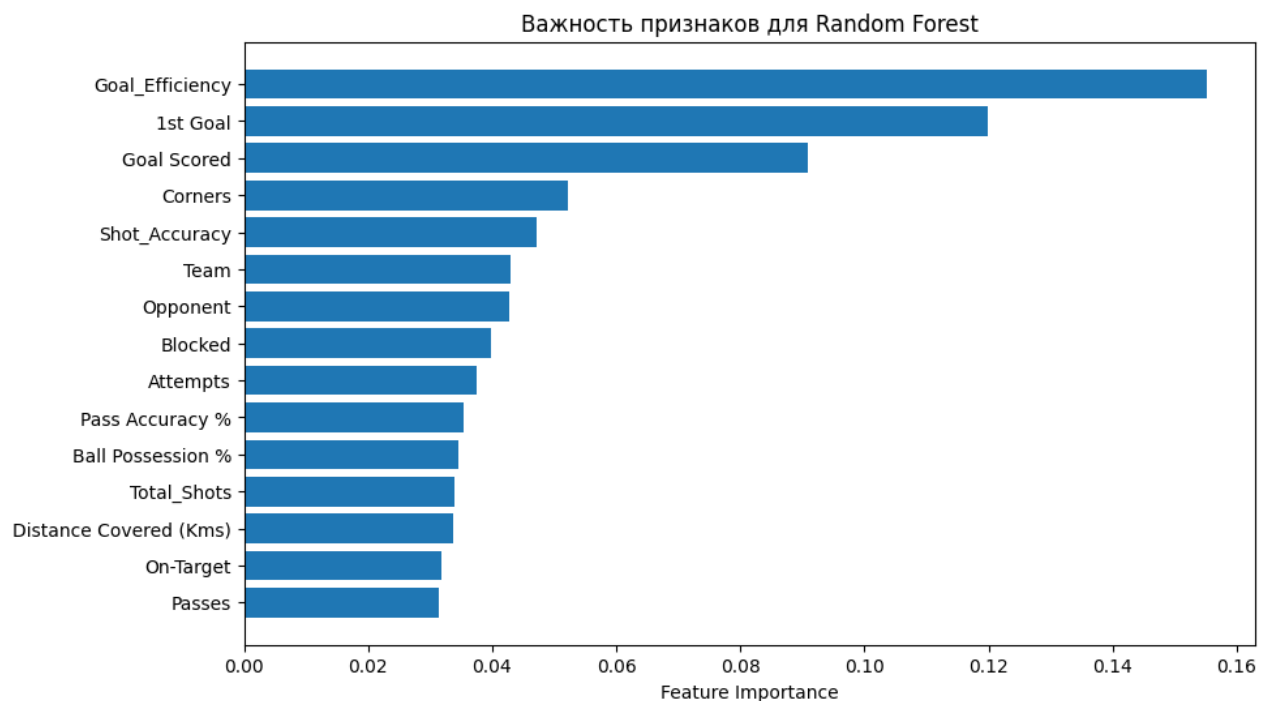
Лучшие гиперпараметры Случайный лес: {'class_weight': 'balanced', 'max_depth': 10, 'min_samples_leaf': 1, 'min_samples_split': 5, 'n_estimators': 500}

Random Forest Performance:
Accuracy: 0.7307692307692307
F1-score: 0.7741935483870968
ROC-AUC: 0.7218934911242603

Classification Report:

	precision	recall	f1-score	support
0	0.88	0.54	0.67	13
1	0.67	0.92	0.77	13
accuracy			0.73	26
macro avg	0.77	0.73	0.72	26
weighted avg	0.77	0.73	0.72	26

Confusion Matrix:
[[7 6]
 [1 12]]



Ассигура (точность) – доля правильных предсказаний.

- Дерево решений: 0,692
- Случайный лес: 0,731

F1-score (F1-мера) – гармоническое среднее precision и recall, полезно при дисбалансе классов.

- Дерево решений: 0,75
- Случайный лес: 0,774

ROC-AUC – площадь под ROC-кривой, показывает способность модели разделять классы.

- Дерево решений: 0,669
- Случайный лес: 0,722

Precision и Recall:

- Precision (точность) – доля верно предсказанных положительных классов.
- Recall (полнота) – доля реальных положительных классов, которые модель нашла.
- У случайного леса выше recall для класса 1 (0,92), но ниже precision (0,67).

Confusion Matrix (матрица ошибок):

- Дерево решений: 6 верных отрицательных (TN), 12 верных положительных (TP), 7 ложных положительных (FP), 1 ложный отрицательный (FN).
- Случайный лес: 7 TN, 12 TP, 6 FP, 1 FN.

Вывод:

1. Случайный лес работает немного лучше, чем дерево решений, по всем метрикам (Accuracy, F1, ROC-AUC).
2. Проблема с предсказанием класса 0 (низкий recall): у дерева решений recall = 0,46, у случайного леса = 0,54: Модели хуже распознают отрицательный класс.